ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN A

AGENT & GIAO THỨC TRUYỀN THÔNG

Phân hệ máy trạm và chuẩn đóng gói dữ liệu mạng cho hệ thống LabGuard

BIÊN SOẠN: VÕ TRỌNG PHÚC

NHÓM THỰC HIỆN: 4 THÀNH VIÊN

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN A

(Đọc xong trang này trong 2 phút là bạn biết chính xác mình cần làm gì hôm nay)

1. BẠN LÀ AI?

Bạn là người phụ trách phần mềm chạy ngầm trên máy sinh viên (**Agent**) và thư viện giao thức đóng gói dữ liệu mạng (**Protocol**).

2. BẠN LÀM GÌ?

- Đọc hiểu thư viện giao thức `src/LabGuard.Protocol/` (đã hoàn thành 100%, bộ kiểm thử giao thức đã PASS).
- Lập trình Agent mở kết nối TCP tới máy chủ Central Server (cổng 5000).
- Định kỳ 5 giây đo tải CPU, RAM và gửi nhịp tim sống còn lên máy chủ.
- Giữ lại số liệu vào bộ nhớ đệm (Replay Buffer) khi bị đứt mạng và tự động gửi bù khi có mạng lại.

3. BẠN KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo việc ghi cơ sở dữ liệu SQL Server hay viết luật phát hiện sự cố (việc của **Thành viên B**).
- Bạn **không cần** lo vẽ giao diện đồ họa WinForms hay bảng điều khiển (việc của **Thành viên C**).
- Ngọc phụ trách Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard; Gateway/Collector chỉ là mở rộng tùy chọn, không phải điều kiện bắt buộc của Agent.

4. HÔM NAY BẮT ĐẦU Ở ĐÂU?

Chạy lệnh `dotnet test tests/LabGuard.Agent.Tests` để thấy toàn bộ 34 bài kiểm thử của Agent màu xanh (16 bài A-003 + 18 bài A-004). Phân hệ Agent đã hoàn thành kết nối mạng và thu thập CPU, RAM, Uptime thật cùng Heartbeat định kỳ. Bước tiếp theo là thực hiện chặng A-005: Tự động kết nối lại (Reconnect với Exponential Backoff).

5. FILES BẠN CẦN MỞ TRƯỚC:

1. `START_HERE.md`: Bản đồ hướng dẫn khởi động nhanh dự án.
2. `src/LabGuard.Protocol/WireEnvelope.cs`: Chiếc phong bì chuẩn 6 trường của toàn hệ thống.
3. `src/LabGuard.Protocol/FrameCodec.cs`: Cách cắt và ghép 4 byte độ dài khi gửi tin qua mạng.
4. `src/LabGuard.Agent/Program.cs`: Launcher async đã có từ PR #11; xem thêm AgentConsoleApplication và AgentSettings.

MỤC LỤC

Hiện trạng và phân công sau PR #11 **1**

1 Tổng quan vai trò của Thành viên A **2**

2 Chặng 1: Thư viện Giao thức (ĐÃ HOÀN THÀNH) **3**
2.1 Cấu trúc chuẩn của WireEnvelope.cs . . . 3
2.2 Bộ đóng khung nhị phân FrameCodec.cs . . . 4
2.3 Giải thuật đọc chính xác N byte (ReadExactlyBytesAsync) . . . 5
2.4 Bộ kiểm tra message đúng luật (EnvelopeValidator.cs) . . . 6
2.5 Bộ chuyển đổi JSON (ProtocolSerializer.cs) . . . 6
2.6 Cấu trúc bộ kiểm thử giao thức thực tế (Protocol Tests) . . . 6

3 Các chặng công việc máy trạm Agent **7**
3.1 ĐÃ HOÀN THÀNH (A-003): Kết nối TCP tới Central và hàng đợi gửi tin an toàn 7
3.2 ĐÃ HOÀN THÀNH (A-004): Thu thập chỉ số tài nguyên hệ thống (MetricsCollector & MetricsService) . . . 9
3.3 ĐÃ HOÀN THÀNH (A-004): Gửi nhịp tim định kỳ duy trì sự sống (HeartbeatService) . . . 10
3.4 Chặng 5: CHƯA LÀM / TIẾP THEO: Bộ đệm ngoại tuyến lưu tạm khi rớt mạng (Replay Buffer) . . . 11
3.5 Chặng 6: CHƯA LÀM / DỰ KIẾN: Giám sát tiến trình phần mềm quy định . . . 13
3.6 Chặng 7: CHƯA LÀM / DỰ KIẾN: Tự động hóa kiểm thử cho các chặng tiếp theo 14

4 Năm bẫy lỗi lập trình cần tránh dành cho Thành viên A **14**

5 Bộ câu hỏi vấn đáp dành riêng cho Thành viên A **16**

Hiện trạng và phân công sau PR #11

TÓM TẮT 30 GIÂY

Mốc nguồn: main `70481fa`, PR #11 đã merge. Đây là cẩm nang học và kế hoạch hiện thực; các mô tả tương lai/pseudocode không phải bằng chứng tính năng đã hoạt động.

- **Agent đã có:** chương trình Console, cấu hình JSON, DeviceId riêng cho mỗi bản cài đặt, đo CPU/RAM/thời gian hoạt động Windows tại máy, chế độ thử TCP/HELLO, hủy tác vụ và ghi nhật ký.
- **Chế độ giám sát bị chặn có chủ đích** tới khi TLS/phiên kết nối/AUTH sẵn sàng. MetricsService/HeartbeatService đã có thành phần và kiểm thử nhưng chưa được bật trong chương trình khởi chạy.
- **Chưa hoàn tất:** A-005 kết nối lại, A-006 ReplayBuffer, đọc phản hồi/kiểm tra mất kết nối, tích hợp SQL/ACK và Windows Service. B-001 đã có bộ tiếp nhận TCP; B-005 mới có DDL, còn đang làm. C-001 có bố cục 0.1, chưa nghiệm thu liên kết dữ liệu và quy trình xử lý.
- **Kiểm thử đã báo cáo từ PR #11:** 195 bài đạt trên Windows tại máy: Protocol 107, Agent 64, Central 9, MockHarness 15. Đây **không phải kết quả GitHub CI**. Số 34/165 trong hồ sơ cũ là mốc lịch sử.

Thành viên	Trách nhiệm chính thức
A — Võ Trọng Phúc	Agent và Protocol.
B — Nguyễn Bá Quang	Central Server và SQL Server.
C — Lý Tuyết Vy	Admin Windows Forms.
D — Đỗ Minh Ngọc	Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard.

Nguồn: quyết định trực tiếp của người điều phối dự án ngày 17/10/2026; không tự tạo biên bản họp hoặc chữ ký. Chức năng hỗ trợ xử lý sự cố là **phần việc chính thức trong kế hoạch**, chưa phải chức năng đã được lập trình. Logic nhúng trong quy trình xử lý sự cố của LabGuard Admin, dùng dữ liệu có thẩm quyền qua Central; D phối hợp B/C, không làm ứng dụng TCP độc lập.

Gateway/Collector và Simulator là **mở rộng P1 tùy chọn** sau khi phần cốt lõi ổn định. Nội dung mở rộng được giữ để học, không phải cam kết chính của Ngọc. GK là một buổi trình bày đề xuất theo lịch giảng viên, không phải hạn hoàn thành đồ án; các chặng là kế hoạch tương đối toàn dự án.

B-009 cần đối chiếu kế hoạch: WBS cũ nhắc AssignedToUserId/IncidentHistory nhưng schema hiện chỉ có AssigneeId và chưa có bảng lịch sử. Giữ bản ghi CHƯA LÀM để Quang/nhóm quyết định, không tự sửa schema hoặc coi bảng/cột cũ đã tồn tại.

Năm mốc khác nhau: kết nối TCP; xác thực ứng dụng; nhận số liệu; ghi thành công SQL; nhận ACK ứng dụng. Chế độ thử trả exit 0 chỉ cho biết lệnh kết thúc; phía bên kia đóng kết nối có thể chưa được phát hiện. Nhật ký kết nối không chứng minh đã xác thực hoặc lưu SQL.

Bắt đầu đúng cách: đọc `docs/agent_deployment/README.md` và `docs/internal/TEAM_ASSIGNMENTS.md`. WBS giữ 34 nhiệm vụ cũ và sáu nhiệm vụ bổ sung cho chức năng hỗ trợ xử lý sự cố, đều chưa làm. Nâng cấp .NET 10 là việc riêng toàn bộ dự án, chưa thực hiện trong lượt đồng bộ này.

1. Tổng quan vai trò của Thành viên A

Launcher đã có: thử đúng mode, không bỏ qua session

```
dotnet run --project src/LabGuard.Agent -- --init-config ./scratch/agent.json
    --central-host 127.0.0.1
dotnet run --project src/LabGuard.Agent -- --config ./scratch/agent.json
    --validate-config
dotnet run --project src/LabGuard.Agent -- --config ./scratch/agent.json
    --collect-once
dotnet run --project src/LabGuard.Agent -- --config ./scratch/agent.json --probe
```

Listing 1: Từ repo root trên Windows; config local, không commit identity

Probe cần Central phát triển được phép, chỉ gửi HELLO không bí mật. Init không ghi đè config; mỗi máy tạo ID riêng. CPU local dùng delta ngắn sau mốc khởi tạo, không phải benchmark. Không chọn mode sẽ dừng monitoring exit 5. Session reader/TLS/AUTH và reconnect/replay vẫn là phần cần phối hợp Quang; không đánh dấu A-005/A-006 XONG vì startup đã có.

TÓM TẮT 30 GIÂY

- Bạn chịu trách nhiệm hai phần: thư viện giao thức chung (**Protocol**) và phần mềm máy trạm (**Agent**).
- Thư viện `Protocol` đã hoàn thành 100% (toàn bộ kiểm thử giao thức đã PASS), là hợp đồng mạng bất biến cho cả nhóm.
- Phần mềm `Agent` đã hoàn thành A-001 đến A-004 (kết nối TCP, hàng đợi Single-Writer, thu thập CPU/RAM/Uptime qua Win32 API và gửi nhịp tim Heartbeat định kỳ 5s). Các phần tiếp theo như reconnect (A-005), replay buffer (A-006) và giám sát dịch vụ (A-007) là kế hoạch tiếp theo chưa triển khai.
- Bạn phối hợp chặt chẽ với Thành viên B (máy chủ tiếp nhận các gói tin của bạn).

Chào mừng bạn đến với phân hệ nền tảng của đồ án LabGuard. Trong nhóm 4 người, bạn là người đặt những viên gạch đầu tiên cho toàn bộ việc kết nối mạng của hệ thống:

KHÁI NIỆM MẠNG CỐT LÕI

Hai khu vực mã nguồn do Thành viên A phụ trách:

1. **Thư viện Giao thức dùng chung (`src/LabGuard.Protocol/`):** Định nghĩa cấu trúc phong bì dữ liệu `WireEnvelope` và bộ đóng khung `FrameCodec`. Đây là hợp đồng mạng bất biến mà cả 3 thành viên còn lại (B, C, D) đều phải dựa vào để trao đổi thông tin. Phân hệ này hiện **ĐÃ HOÀN THÀNH 100%** với bộ kiểm thử tự động đã PASS.
2. **Phần mềm Máy trạm Agent (`src/LabGuard.Agent/`):** Lập trình ứng dụng chạy ngầm trên từng máy tính sinh viên, định kỳ đo CPU/RAM và phát `HEARTBEAT`; Reconnect/ReplayBuffer là các chặng tiếp theo chưa triển khai. **Tác vụ A-003 và A-004 đã HOÀN THÀNH & NGHIỆM THU (34/34 tests ở mốc A-004 (lịch sử), đối soát Windows 10/10 PASS).** Tác vụ A-005+ (Reconnect, ReplayBuffer) là **CHƯA LÀM — TIẾP THEO** để bạn tự tay xây dựng.

Hình này chỉ cần nhìn theo mũi tên từ dưới lên trên: Agent sử dụng thư viện Protocol chung của dự án.

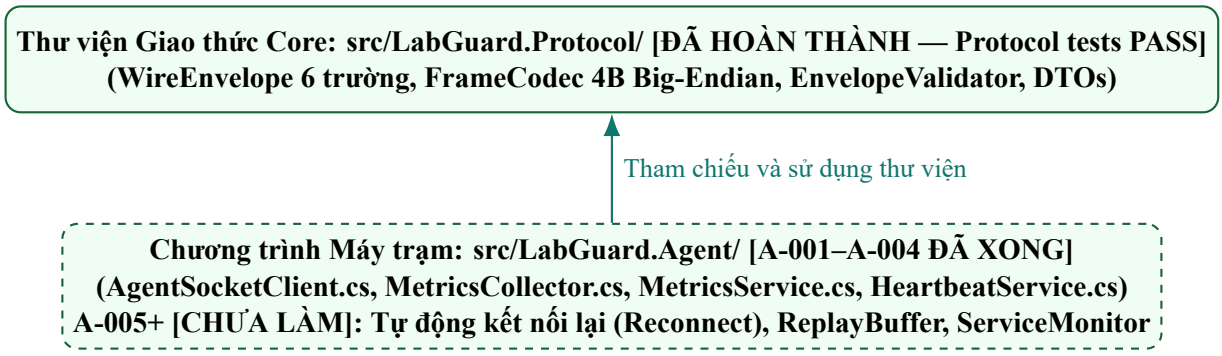


Figure 1: Sơ đồ mối quan hệ giữa 2 phân hệ do Thành viên A phụ trách

Ý chính: Thư viện Protocol đã hoàn thành 100% với bộ kiểm thử giao thức đã PASS, đóng vai trò nền móng bất biến để bạn yên tâm lập trình Agent.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Thư viện Protocol là tài sản chung của cả nhóm, tuyệt đối không tự ý đổi cấu trúc 6 trường phong bì.
- Việc tiếp theo của bạn tập trung 100% vào thư mục `src/LabGuard.Agent/`.

2. Chặng 1: Thư viện Giao thức & Đóng khung tin (ĐÃ HOÀN THÀNH)

TÓM TẮT 30 GIÂY

- Dữ liệu gửi qua mạng luôn bỏ vào phong bì `WireEnvelope` đúng 6 trường.
- Trước khi gửi qua socket TCP, thêm 4 byte độ dài Big-Endian (`FrameCodec`).
- Khi nhận dữ liệu, phải dùng vòng lặp gom đủ byte (`ReadExactlyBytesAsync`), không được đọc nửa vời.
- Đã có sẵn bộ kiểm thử tự động PASS chứng minh thư viện chạy đúng 100%.

GHI CHÚ QUAN TRỌNG

Đây là phần việc **ĐÃ HOÀN TẤT** trong mã nguồn thực tế. Toàn bộ 107 bài test tự động trong `tests/LabGuard.Protocol.Tests` đều đã chạy thành công mỹ mãn. Bạn cần đọc hiểu sâu sắc phần này để tự tin khi viết mã cho Agent và khi trả lời vấn đáp.

2.1 Cấu trúc chuẩn của WireEnvelope.cs

Mọi dữ liệu truyền qua mạng trong LabGuard đều được đặt vào một chiếc ”phong bì” JSON có **CHÍNH XÁC 6 TRƯỜNG CỐ ĐỊNH**. Dưới đây là trích xuất từ file `src/LabGuard.Protocol/WireEnvelope.cs`:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- 6 trường cố định JSON: `version`, `messageType`, `requestId`, `deviceId`, `sentAtUtc`, `payload`.
- Hàm tĩnh `Create()` giúp tự động điền Version = 1, sinh UUID ngẫu nhiên và lấy giờ UTC chuẩn.

```
// Trich doan tu ma nguon thuc te: 6 thuoc tinh phong bi du lieu chuan
public class WireEnvelope<T>
{
    [JsonPropertyName("version")]     public int Version { get; set; } = 1;
    [JsonPropertyName("messageType")] public string MessageType { get; set; } =
        string.Empty;
    [JsonPropertyName("requestId")]   public string RequestId { get; set; } =
        string.Empty;
    [JsonPropertyName("deviceId")]    public string DeviceId { get; set; } =
        string.Empty;
    [JsonPropertyName("sentAtUtc")]   public string SentAtUtc { get; set; } =
        string.Empty;
    [JsonPropertyName("payload")]     public T Payload { get; set; } = default!;

    // Phuong thuc Create(...) tien ich khoi tao nhanh (xem file nguon day du)
    public static WireEnvelope<T> Create(string type, string devId, T payload,
        string? reqId = null) => ...;
}
```

Listing 2: Ví dụ rút gọn dựa trên mã nguồn hiện tại: src/LabGuard.Protocol/WireEnvelope.cs

Ý nghĩa 6 trường phong bì:

- `version`: Số nguyên 1. Tuyệt đối không dùng dạng chuỗi `"1.0"`.
- `messageType`: Tên thông điệp viết hoa (ví dụ: `HEARTBEAT`, `METRIC_BATCH`).
- `requestId`: Mã UUID v4 duy nhất cho mỗi lượt gửi (36 ký tự).
- `deviceId`: Tên máy gửi bản tin (ví dụ: `PC-LAB1-01` hoặc `LAB01-PC02`).
- `sentAtUtc`: Thời điểm gửi theo giờ UTC quốc tế, kết thúc bằng Ź hoặc `+00:00`.
- `payload`: Dữ liệu chi tiết của từng loại thông điệp (tham số generic `T`, như `JsonElement` hoặc DTO cụ thể).

2.2 Bộ đóng khung nhị phân FrameCodec.cs

TCP là dòng byte liên tục (*Byte Stream*), không có khái niệm gói tin nguyên vẹn. Lớp `FrameCodec.cs` giải quyết vấn đề này bằng cách thêm 4 byte độ dài Big-Endian vào đầu mỗi chuỗi JSON trước khi truyền qua socket:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Tiêu đề 4 byte Big-Endian chứa kích thước chính xác của mảng byte payload.
- Hạn mức an toàn `MaxPayloadSize = 262_144` (256 KiB) giúp bảo vệ hệ thống khỏi tràn bộ nhớ.

```
// Trich doan: Hang so co dinh 4 byte do dai va gioi han 256 KiB
public static class FrameCodec
{
```

```
    public const int HeaderSize = 4;                                    // 4 byte do dai Big-Endian
    public const int MaxPayloadSize = 262_144;                          // 256 KiB
    public const int MaxWireFrameSize = HeaderSize + MaxPayloadSize; // 262,148 bytes
}
```

Listing 3: Ví dụ rút gọn dựa trên mã nguồn hiện tại: Hằng số giới hạn trong FrameCodec.cs

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `BinaryPrimitives.WriteUInt32BigEndian` ghi đúng thứ tự byte mạng chuẩn mực.
- Nối liền 4 byte độ dài với mảng byte nội dung JSON phía sau.

```
// Trich doan: Dong goi 4 byte Big-Endian vao dau mang byte
public static byte[] Encode(ReadOnlyMemory<byte> payload)
{
    if (payload.Length == 0 || payload.Length > MaxPayloadSize)
        throw new ArgumentException("Kich thuoc payload khong hop le.");

    byte[] frame = new byte[HeaderSize + payload.Length];
    BinaryPrimitives.WriteUInt32BigEndian(frame.AsSpan(0, HeaderSize),
        (uint)payload.Length);
    payload.Span.CopyTo(frame.AsSpan(HeaderSize));
    return frame;
}
```

Listing 4: Ví dụ rút gọn dựa trên mã nguồn hiện tại: Phương thức Encode trong FrameCodec.cs

2.3 Giải thuật đọc chính xác N byte (ReadExactlyBytesAsync)

Khi nhận dữ liệu từ mạng, một lần gọi `stream.ReadAsync` có thể chỉ trả về một vài byte đầu tiên do phân mảnh mạng. Phương thức `ReadExactlyBytesAsync` dùng vòng lặp tích lũy để gom đủ từng byte:

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Vòng lặp `while (totalRead < count)` tiếp tục đọc cho tới khi nhận đủ số byte yêu cầu.
- Nếu kết quả `read == 0`, nghĩa là đối phương đã chủ động ngắt kết nối mạng.

```
// Trich doan: Vong lap gom tich luy du count byte tu NetworkStream
public static async Task ReadExactlyBytesAsync(
    Stream stream, byte[] buffer, int offset, int count, CancellationToken ct = default)
{
    int totalRead = 0;
    while (totalRead < count)
    {
        int read = await stream.ReadAsync(buffer.AsMemory(offset + totalRead,
            count - totalRead), ct);
        if (read == 0)
            throw new EndOfStreamException($"Mat ket noi TCP: da doc
                {totalRead}/{count} bytes.");
        totalRead += read;
    }
}
```

Listing 5: Ví dụ rút gọn dựa trên mã nguồn hiện tại: ReadExactlyBytesAsync trong FrameCodec.cs

2.4 Bộ kiểm tra message đúng luật (EnvelopeValidator.cs)

Lớp `EnvelopeValidator.cs` thi hành nguyên tắc bảo mật: *"Không bao giờ tin tưởng dữ liệu thô từ mạng"*. Mỗi bản tin sau khi giải mã JSON đều được kiểm tra:

- Kiểm tra `Version == 1`.
- Kiểm tra `MessageType` không rỗng và thuộc danh mục 69 loại chuẩn trong `MessageTypes.cs`.
- Kiểm tra `RequestId` là chuỗi GUID 36 ký tự hợp lệ.
- Kiểm tra `DeviceId` không được rỗng và không chứa ký tự nguy hiểm.
- Kiểm tra `SentAtUtc` có chỉ định múi giờ UTC tường minh (`Ź` hoặc `+00:00`).

2.5 Bộ chuyển đổi JSON (ProtocolSerializer.cs)

Lớp `ProtocolSerializer.cs` cấu hình thư viện `System.Text.Json` với các tùy chọn chuẩn hóa giúp chuỗi JSON luôn gọn gàng (minified) và không tốn byte thừa trên đường truyền.

2.6 Cấu trúc bộ kiểm thử giao thức thực tế (Protocol Tests)

Thư viện kiểm thử `tests/LabGuard.Protocol.Tests` bao gồm 107 bài test tự động:

1. **`FrameCodecTests.cs`:** Kiểm thử đóng khung 4 byte Big-Endian, đọc phân mảnh TCP, phát hiện khung quá khổ `MaxPayloadSize` (256 KiB).
2. **`ProtocolSpecificationTests.cs`:** Kiểm thử tuân thủ đặc tả giao thức, kiểm tra 6 trường outer, xác thực giờ UTC.
3. **`WireContractAndFixturesTests.cs`:** Kiểm thử khớp 16 fixture JSON quy chuẩn và tuần tự hóa hai chiều.
4. **`ChunkedStream.cs`:** Luồng giả lập ngắt quãng từng byte để thử nghiệm khả năng chịu phân mảnh mạng.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Thư viện Protocol đã pass toàn bộ 107/107 tests tự động.
- Khi làm Agent, bạn chỉ việc gọi `FrameCodec.Encode(env)` để gửi và `FrameCodec.ReadFrameAsync(stream)` để nhận.

3. Chặng A-003 & A-004 ĐÃ XONG; Chặng A-005+ (Chưa làm / Dự kiến)

TÓM TẮT 30 GIÂY

- Agent chạy ngầm trên máy sinh viên, kết nối TCP tới Central Server (Port 5000).
- Dùng hàng đợi một luồng ghi (`Single-Writer Channel`) để gửi tin an toàn không bị trộn byte (A-003 ĐÃ XONG).
- Định kỳ gửi `HEARTBEAT` và `METRIC_BATCH` qua `PeriodicTimer` 5s (A-004 ĐÃ XONG).
- A-005 sẽ bổ sung tự động kết nối lại (Reconnect) với Exponential Backoff.
- A-006 dự kiến bổ sung ReplayBuffer tối đa 50 METRIC_BATCH.

GHI CHÚ QUAN TRỌNG

Chặng A-003 & A-004 ĐÃ HOÀN THÀNH: `AgentSocketClient.cs` (BoundedChannel 256, Single-Writer Loop), `MetricsCollector.cs` (P/Invoke Win32: CPU, RAM, Uptime), `MetricsService.cs`, `HeartbeatService.cs` — Tổng cộng 34/34 tests Agent PASS, đối soát tự động Windows reference 10/10 PASS.
Các chặng tiếp theo từ A-005 trở đi đang ở trạng thái **CHƯA LÀM / TIẾP THEO**. Bạn sẽ tiếp tục lập trình trong thư mục `src/LabGuard.Agent/`.

3.1 ĐÃ HOÀN THÀNH (A-003): Kết nối TCP tới Central và hàng đợi gửi tin an toàn

Chặng 2 — Mã tác vụ nội bộ: LG-A-003 — XONG (16/16 tests PASS)

3.1.1 Tại sao cần phần này?

Trong một chương trình Agent, có hai việc xảy ra đồng thời: một luồng định kỳ gửi nhịp tim sống còn (`HEARTBEAT` mỗi 5s), một luồng khác định kỳ gửi số liệu CPU/RAM (`METRIC_BATCH` mỗi 5s). Nếu cả hai luồng cùng gọi thẳng hàm ghi vào socket TCP (`stream.WriteAsync`), các byte của hai gói tin sẽ bị trộn lẫn vào nhau, khiến máy chủ không thể đọc được và ngắt kết nối ngay!

3.1.2 Khi chạy, chuyện gì xảy ra?

Agent khởi tạo `TcpClient` kết nối đến Central Server trên cổng 5000. Để đảm bảo an toàn tuyệt đối mà không cần dùng khóa chặn nghẽn (`lock`), ta dùng một hàng đợi có giới hạn dung lượng (**Bounded Single-Writer Queue**) bằng `Channel<byte[]>`:

- Các luồng nghiệp vụ (Heartbeat, đo CPU/RAM) chỉ việc đẩy mảng byte vào hàng đợi.
- Duy nhất một luồng nền độc lập lấy từng gói tin ra và ghi tuần tự vào `NetworkStream`.

Sơ đồ luồng gửi tin tuần tự qua hàng đợi Single-Writer trong `AgentSocketClient`:

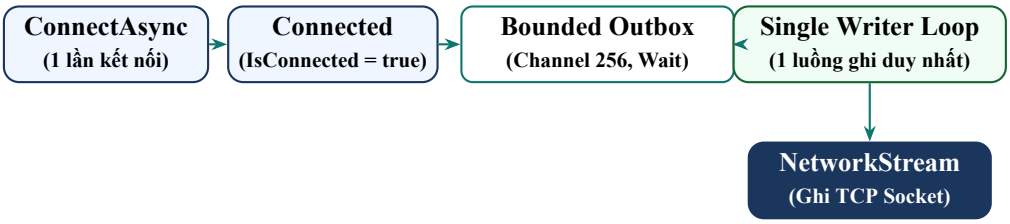


Figure 2: Sơ đồ luồng gửi tin qua hàng đợi Single-Writer trong AgentSocketClient (A-003)

Ý chính: `AgentSocketClient` thực hiện một lần kết nối duy nhất (`ConnectAsync`); nếu thất bại, ngoại lệ được chuyển tiếp lên cho bên gọi xử lý. Khi xảy ra lỗi ghi mạng, client lập tức đánh dấu `IsConnected = false` và dừng luồng outbox an toàn. Cơ chế tự động kết nối lại (Reconnect) thuộc nhiệm vụ A-005.

3.1.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/AgentSocketClient.cs`.

3.1.4 Bạn cần làm những bước lớn nào?

1. Mở kết nối TCP tới máy chủ bằng `TcpClient.ConnectAsync(host, port)`.
2. Khởi tạo hàng đợi có giới hạn `BoundedChannelOptions(256)` với chế độ chờ `FullMode.Wait` để bộ nhớ không tăng vô hạn.
3. Khởi chạy vòng lặp gửi tin đơn lẻ (Single Writer Loop) đọc từ Channel và ghi vào socket.
4. Quản lý vòng đời kết nối, xử lý lỗi ghi mạng và giải phóng tài nguyên an toàn qua `DisposeAsync`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Kết nối tới máy chủ và duy trì luồng mạng `NetworkStream`.
- Sử dụng hàng đợi Bounded Channel 256 ngăn ngừa tranh chấp luồng mạng và bảo vệ bộ nhớ RAM.

```
// Pseudocode minh hoa: Single-Writer Loop trong AgentSocketClient
private async Task WriterLoopAsync(CancellationToken ct)
{
    while (await _outbox.Reader.WaitToReadAsync(ct))
    {
        while (_outbox.Reader.TryRead(out var frame))
        {
            await _stream.WriteAsync(frame, ct);
        }
    }
}
```

Listing 6: Pseudocode minh họa: Vòng lặp gửi tin đơn lẻ Single-Writer Loop

3.1.5 Làm sao thử?

Chạy bộ kiểm thử tự động 16/16 bài trong `AgentSocketClientTests.cs` với máy chủ giả lập `FakeTcpServer`: kiểm chứng kết nối TCP, gửi đơn khung, gửi tuần tự, nhiều producer đồng thời, backpressure hàng đợi 256 items, hủy thao tác (cancellation), và ngắt kết nối an toàn khi gặp sự cố mạng.

3.1.6 Khi nào coi là xong? (PASS Criteria)

- Kết nối thành công tới Central Server trên cổng 5000 qua `ConnectAsync` (1 lần thử kết nối).
- Hàng đợi `BoundedChannelOptions(256)` với `FullMode.Wait` điều phối an toàn khi nhiều luồng cùng gửi tin.
- Duy nhất một luồng Single Writer Loop ghi tuần tự vào `NetworkStream`, phát hiện lỗi ghi và chuyển `IsConnected = false`.
- Vượt qua toàn bộ 16/16 bài kiểm thử tự động của `AgentSocketClientTests` (lưu ý: tính năng tự động kết nối lại thuộc nhiệm vụ A-005).

3.2 ĐÃ HOÀN THÀNH (A-004): Thu thập chỉ số tài nguyên hệ thống (MetricsCollector & MetricsService)

Chặng 3 — Mã tác vụ: LG-A-004 — ĐÃ NGHIỆM THU (10/10 MetricsCollector tests PASS, 4/4 MetricsService tests PASS)

3.2.1 Tại sao cần phần này?

Mục đích chính của LabGuard là giám sát tình trạng hoạt động của phòng máy: máy tính có bị treo đơ, quá tải CPU hay cạn kiệt bộ nhớ RAM hay không. Để bảo đảm số liệu phản ánh trung thực toàn hệ thống và tiêu tốn cực ít tài nguyên, đạt yêu cầu WBS < 100ms, Agent gọi trực tiếp các hàm Windows API cấp thấp thay vì dùng PerformanceCounter cồng kềnh.

3.2.2 Khi chạy, chuyện gì xảy ra?

Định kỳ mỗi 5 giây, lớp `MetricsService` kích hoạt một chu kỳ thu thập tài nguyên:

- **Tải CPU toàn hệ thống (%):** Gọi Win32 API `GetSystemTimes` qua P/Invoke, tính tỷ lệ chênh lệch thời gian rỗi (`idleDelta`) so với tổng thời gian hệ thống (`kernelDelta + userDelta`) giữa hai lần lấy mẫu.
- **Bộ nhớ RAM vật lý:** Gọi Win32 API `GlobalMemoryStatusEx` lấy tỷ lệ sử dụng RAM (%), dung lượng RAM khả dụng (`ullAvailPhys`) và dung lượng RAM đang sử dụng (`ullTotalPhys - ullAvailPhys`) quy đổi ra Megabyte (MB).
- **Thời gian hoạt động (Uptime):** Gọi Win32 API `GetTickCount64` chia cho 1000 để lấy số giây máy đã bật.

Số liệu được đóng gói vào đối tượng chuẩn `MetricBatchPayload` với `bootId` ổn định suốt vòng đời chương trình, sinh mới `batchId` ngẫu nhiên (GUID) cho mỗi đợt, mã hóa thành `WireEnvelope` và gửi qua phương thức `AgentSocketClient.SendEnvelopeAsync(...)` (đưa vào kênh outbox nội bộ).

3.2.3 Các file mã nguồn thực tế

- `src/LabGuard.Agent/MetricsCollector.cs`: Định nghĩa giao diện `IMetricsCollector` và cài đặt `WindowsMetricsCollector` bằng Win32 P/Invoke.
- `src/LabGuard.Agent/MetricsService.cs`: Dịch vụ định thời `PeriodicTimer(5s)`, thu thập và gửi bản tin `METRIC_BATCH` qua `SendEnvelopeAsync(...)`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `GetSystemTimes` đo chính xác mức tải CPU của toàn bộ máy tính (System-wide), không bị giới hạn trong tiến trình đơn lẻ.
- `GlobalMemoryStatusEx` đọc trực tiếp RAM vật lý của hệ điều hành, bảo đảm độc lập tuyệt đối với Garbage Collector.
- Đạt yêu cầu WBS < 100ms. Trong phiên validation 10 mẫu, độ trễ tối đa ghi nhận 0.616ms (mẫu warm-up ban đầu ghi nhận 1.137ms, các mẫu tiếp theo từ 0.026 đến 0.332ms), cách xa ngưỡng nghiệm thu < 100ms trong các phép đo đã thực hiện.

3.2.4 Kết quả nghiệm thu kỹ thuật (PASS Criteria)

- $14/14$ bài kiểm thử đơn vị và tích hợp của bộ đo vượt qua 100% ($10/10$ `Metrics CollectorTests`, $4/4$ `MetricsServiceTests`).
- Đối soát tự động với nguồn đối chuẩn độc lập của Windows ($10/10$ mẫu đo PASS): sai lệch CPU tối đa chỉ $\approx$ 0.305 điểm phần trăm (pp), sai lệch RAM tối đa chỉ $\approx$ 0.125 điểm phần trăm (pp).

3.3 ĐÃ HOÀN THÀNH (A-004): Gửi nhịp tim định kỳ duy trì sự sống (HeartbeatService)

Chặng 4 — Mã tác vụ: LG-A-004 — ĐÃ NGHIỆM THU (4/4 HeartbeatService tests PASS)

3.3.1 Tại sao cần phần này?

Nếu một máy tính sinh viên bị tắt nguồn đột ngột hoặc rút dây mạng, máy tính đó không kịp gửi bất kỳ lời chào tạm biệt nào. Gói tin nhịp tim gửi đều đặn mỗi 5 giây giúp máy chủ Central duy trì bảng theo dõi sống còn (Liveness Tracker). Nếu sau 20 giây (> 20s) Central không nhận được tin, máy sẽ bị coi là mất kết nối.

3.3.2 Khi chạy, chuyện gì xảy ra?

Định kỳ với chu kỳ cấu hình mặc định 5 giây, lớp `HeartbeatService` dùng `PeriodicTimer` phát một bản tin `HEARTBEAT` siêu nhẹ:

- Số thứ tự `SequenceNumber` tăng đơn điệu liên tục $(1, 2, 3, \ldots)$.
- Thời gian hoạt động `UptimeSeconds` lấy từ `IMetricsCollector.ReadUptimeSeconds()`.
- Bản tin được gửi qua `AgentSocketClient.SendEnvelopeAsync(...)` (mã hóa và đưa vào kênh outbox nội bộ).
- Nếu kết nối mạng gián đoạn, kênh đóng hoặc token bị hủy, vòng lặp nhịp tim tự động kết thúc an toàn.

3.3.3 Mã nguồn thực tế

File: `src/LabGuard.Agent/HeartbeatService.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `PeriodicTimer` cung cấp vòng lặp định kỳ bất đồng bộ thuận tiện, tránh busy-loop và ngăn callback chồng lấn trong thiết kế hiện tại. Scheduler delay của hệ điều hành hoặc công việc kéo dài vẫn có thể ảnh hưởng thời gian thực tế.
- `SequenceNumber` bắt đầu từ 1 và tăng dần đơn điệu, không bao giờ bị nhảy cóc hay lặp lại.
- Gửi dữ liệu qua phương thức `SendEnvelopeAsync(...)` tới kênh outbox nội bộ của `AgentSocketClient`, giải phóng tài nguyên sạch sẽ khi dừng dịch vụ.

3.3.4 Kết quả nghiệm thu kỹ thuật (PASS Criteria)

- 4/4 bài kiểm thử tự động của `HeartbeatServiceTests` vượt qua 100% (kiểm thử phát nhịp tim tuần tự, tăng sequence, định dạng phong bì chuẩn và hủy vòng lặp đúng lúc).
- `HeartbeatService` định kỳ tạo và enqueue `HEARTBEAT` qua `AgentSocketClient` khi kết nối đang hoạt động; việc phục hồi kết nối sau lỗi mạng thuộc nhiệm vụ A-005.

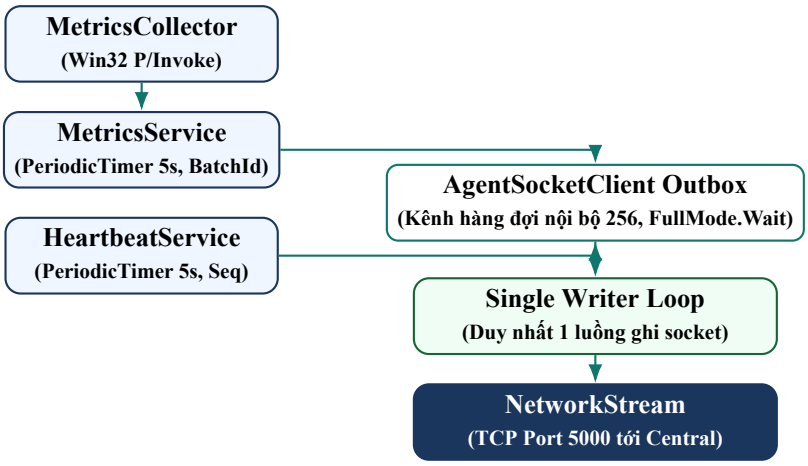


Figure 3: Luồng xử lý dữ liệu trong các thành phần Agent của A-003 và A-004

3.4 Chặng 5: CHƯA LÀM / TIẾP THEO: Bộ đệm ngoại tuyến lưu tạm khi rớt mạng (Replay Buffer)

Chặng 5 — Mã tác vụ: LG-A-006 — CHƯA LÀM / DỰ KIẾN (Sau khi hoàn thành A-005 Reconnect)

3.4.1 Tại sao cần phần này?

Khi sinh viên vô tình đá phải chân cáp mạng, dây mạng bị lỏng trong chốc lát. Trong khoảng thời gian đó, Agent vẫn đo CPU/RAM nhưng không gửi đi được. Nếu vứt bỏ số liệu thì máy chủ sẽ bị mất dữ liệu lịch sử.

3.4.2 Khi chạy, chuyện gì xảy ra?

Agent lưu các mẻ số liệu đo đạc chưa gửi được vào một hàng đợi trong bộ nhớ RAM (`ReplayBuffer`). Hàng đợi này chứa tối đa 50 mẻ số liệu (`METRIC_BATCH`, tương đương 250 giây dữ liệu), tự động loại bỏ mẻ cũ nhất nếu đầy theo chính sách FIFO Drop-Oldest để không làm tràn

RAM. Lưu ý: Chỉ lưu `METRIC_BATCH`, tuyệt đối không lưu `HEARTBEAT` hay `SERVICE_STATUS` vào bộ đệm vì dữ liệu thời gian thực cũ không còn giá trị liveness. Khi kết nối mạng có lại, Agent tự động gửi bù các mẻ này kèm cờ `isReplay = true`.

3.4.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/ReplayBuffer.cs`.

3.4.4 Bạn cần làm những bước lớn nào?

1. Sử dụng hàng đợi an toàn luồng `ConcurrentQueue<byte[]>`.
2. Chỉ đưa bản tin `METRIC_BATCH` vào bộ đệm; nếu số lượng vượt quá 50 mẻ thì loại bỏ mẻ cũ nhất (`Drop Oldest`).
3. Viết hàm `DrainAsync` để đẩy bù dữ liệu khi mạng phục hồi.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Giới hạn dung lượng 50 mẻ ngăn ngừa lỗi tràn bộ nhớ (OutOfMemoryException), cờ `isReplay = true` thông báo dữ liệu gửi bù.
- Xả hàng đợi theo trật tự Vào trước - Ra trước (FIFO) để bảo đảm dòng thời gian số liệu.

```
// Pseudocode minh hoa: Bo dem hang doi luu tam va phat lai
public class ReplayBuffer
{
    private readonly ConcurrentQueue<byte[]> _queue = new();
    public void Enqueue(byte[] frame)
    {
        if (_queue.Count >= 50) _queue.TryDequeue(out _); // Giu toi da 50 me (250s)
        _queue.Enqueue(frame);
    }
    public async Task DrainAsync(Func<byte[], Task> sendFunc)
    {
        while (_queue.TryDequeue(out var frame)) await sendFunc(frame);
    }
}
```

Listing 7: Pseudocode minh họa: Bộ đệm hàng đợi lưu tạm và phát lại

3.4.5 Làm sao thử?

Chạy Agent, rút dây mạng ra 10 giây rồi cắm lại: kiểm tra log thấy Agent tự động đẩy bù các gói tin đã gom trong thời gian mất mạng.

3.4.6 Khi nào coi là xong? (PASS Criteria)

- Bộ đệm không bao giờ vượt quá 50 mẻ dữ liệu (chỉ lưu METRIC_BATCH, cờ isReplay=true).
- Bảo toàn thứ tự vào trước ra trước (FIFO).
- Tự động xả sạch bộ đệm ngay khi kết nối TCP được tái thiết lập.

3.5 Chặng 6: CHƯA LÀM / DỰ KIẾN: Giám sát tiến trình phần mềm quy định

Chặng 6 — Mã tác vụ: LG-A-007 — CHƯA LÀM / DỰ KIẾN

3.5.1 Tại sao cần phần này?

Trong phòng thi thực hành, có những phần mềm bắt buộc phải chạy ngầm (ví dụ phần mềm thi cử). Ta cần phát hiện ngay lập tức nếu sinh viên vô tình hoặc cố ý tắt phần mềm này.

3.5.2 Khi chạy, chuyện gì xảy ra?

Agent định kỳ kiểm tra danh sách tiến trình đang hoạt động trên máy tính. Nếu thiếu tiến trình bắt buộc, Agent gửi ngay cảnh báo sự cố lên Central Server.

3.5.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Agent/ServiceMonitor.cs`.

3.5.4 Bạn cần làm những bước lớn nào?

1. Đọc danh sách tiến trình qua `Process.GetProcesses()`.
2. So khớp tên không phân biệt hoa thường với danh sách bắt buộc.
3. Trả về danh sách các dịch vụ bị thiếu.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- So sánh tên tiến trình không phân biệt hoa thường (`OrdinalIgnoreCase`).
- Trả về danh sách các tiến trình bị tắt để gửi cảnh báo.

```
// Pseudocode minh hoa: Kiem tra tien trinh quy dinh
public class ServiceMonitor
{
    public List<string> CheckMissing(IEnumerable<string> required)
    {
        var running = Process.GetProcesses().Select(p =>
            p.ProcessName).ToHashSet(StringComparer.OrdinalIgnoreCase);
        return required.Where(req => !running.Contains(req)).ToList();
    }
}
```

Listing 8: Pseudocode minh họa: Kiểm tra phần mềm quy định có đang chạy không

3.5.5 Làm sao thử?

Cấu hình theo dõi tiến trình `notepad`, bật Notepad lên rồi đóng lại: Agent lập tức in log cảnh báo phát hiện tiến trình bị tắt.

3.5.6 Khi nào coi là xong? (PASS Criteria)

- Nhận diện chính xác tiến trình bị tắt trong vòng 1 chu kỳ kiểm tra.
- So khớp tên không phân biệt chữ hoa chữ thường.

3.6 Chặng 7: CHƯA LÀM / DỰ KIẾN: Tự động hóa kiểm thử cho các chặng tiếp theo

Mã tác vụ nội bộ: LG-A-008, LG-A-009 — CHƯA LÀM / DỰ KIẾN

3.6.1 Tại sao cần phần này?

Viết kiểm thử tự động giúp bạn chắc chắn rằng code Agent của mình chạy ổn định, không bị lỗi bộ nhớ hay lỗi logic trước khi nộp bài cho thầy cô.

3.6.2 Khi chạy, chuyện gì xảy ra?

Lệnh `dotnet test tests/LabGuard.Agent.Tests` sẽ chạy qua các tình huống giả lập: mất mạng, đầy bộ đệm, đọc sai khung tin, và xác nhận kết quả Passed.

3.6.3 Bạn sẽ đụng file nào?

Tạo các tệp kiểm thử trong: `tests/LabGuard.Agent.Tests/`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Bơm 55 mẻ số liệu vào bộ đệm dung lượng 50.
- Khẳng định số lượng phần tử còn lại không bao giờ vượt quá 50 mẻ.

```
// Vi du rut gon: Kiem tra bo dem khong vuot qua 50 me (chi METRIC_BATCH)
[Fact]
public void ReplayBuffer_WhenExceeded_DropsOldest()
{
    var buffer = new ReplayBuffer();
    for (int i = 0; i < 55; i++) buffer.Enqueue(new byte[] { 1, 2, 3 });
    Assert.True(buffer.Count <= 50);
}
```

Listing 9: Ví dụ rút gọn: Bài kiểm thử đơn vị cho bộ đệm ReplayBuffer

3.6.4 Khi nào coi là xong? (PASS Criteria)

- Lệnh `dotnet test tests/LabGuard.Agent.Tests` chạy hoàn tất thành công.
- 100% các ca kiểm thử đạt kết quả Passed (Failed: 0).

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- 6 chặng của Agent được xây dựng tuần tự: kết nối TCP → đo CPU/RAM → gửi Heartbeat → đệm Replay → giám sát tiến trình → kiểm thử tự động.
- Mọi gói tin đều đi qua `FrameCodec.Encode()` trước khi gửi và kiểm tra an toàn luồng.

4. Năm bẫy lỗi lập trình cần tránh dành cho Thành viên A

Dưới đây là 5 lỗi kỹ thuật phổ biến nhất kèm theo ví dụ so sánh mã nguồn giữa cách viết sai và cách viết chuẩn mực:

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Quên chuyển đổi Big-Endian cho độ dài khung tin
Nếu bạn dùng `BitConverter.GetBytes(length)` trên máy tính chip Intel/AMD (kiến trúc Little-Endian), số nguyên sẽ bị lưu ngược thứ tự byte. Máy chủ đọc vào sẽ ra con số hàng triệu, làm sập chương trình!

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Không dùng `BitConverter` vì phụ thuộc vào chip phần cứng.
- Luôn dùng `BinaryPrimitives.WriteUInt32BigEndian` để bảo đảm chuẩn mạng.

```
// [SAI] Dung BitConverter tren may x86 gay dao nguoc byte:
byte[] prefix = BitConverter.GetBytes(payloadLength); // Nguy hiem!

// [DUNG] Bat buoc dung BinaryPrimitives de dam bao Big-Endian:
byte[] prefix = new byte[4];
BinaryPrimitives.WriteUInt32BigEndian(prefix, (uint)payloadLength); // Chuan mang
```

Listing 10: So sánh Bẫy 1: Little-Endian vs Big-Endian chuẩn

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Tràn bộ nhớ RAM do lưu đệm vô tận khi mất mạng (Thiết kế A-006)
Trong thiết kế hàng đợi lưu đệm ngoại tuyến (A-006 dự kiến), nếu mất mạng lâu mà Agent cứ nhét gói tin vào bộ nhớ không giới hạn, chương trình sẽ ngốn sạch RAM máy tính sinh viên (*Out Of Memory*). Cần đặt giới hạn trần cho bộ đệm (50 mẻ METRIC_BATCH) và loại bỏ mẻ cũ nhất khi đầy (FIFO Drop-Oldest).

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Dùng hàng đợi không giới hạn sẽ làm cạn kiệt RAM khi rớt mạng lâu.
- Luôn có điều kiện kiểm tra `Count >= 50` và loại bỏ phần tử cũ nhất.

```
// [SAI] Hang doi khong gioi han lam tran bo nho RAM:
_unboundedQueue.Enqueue(packet); // Co the ngon hang Gigabyte RAM neu mat mang
    lau!

// [DUNG] Bounded Queue co co che go bo ban tin cu nhat:
while (_boundedQueue.Count >= 50) _boundedQueue.TryDequeue(out _);
_boundedQueue.Enqueue(packet);
```

Listing 11: So sánh Bẫy 2: Hàng đợi vô hạn vs Bounded FIFO Queue

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Đọc Socket bằng ReadAsync mà không đọc lặp
Một lần gọi `ReadAsync` không bảo đảm trả về đủ khung tin. Nếu bạn lấy luôn dữ liệu chưa đủ byte để giải mã JSON, hệ thống sẽ ném ngoại lệ `JsonException`. Luôn dùng vòng lặp tích lũy byte như trong `FrameCodec.ReadExactlyBytesAsync`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Thử kết nối lại socket quá nhanh khi cài đặt Reconnect (A-005)
Khi thực hiện tác vụ A-005 sắp tới, nếu dùng vòng lặp `while(true)` gọi `ConnectAsync` liên tục không có độ trễ nghỉ (`Task.Delay`), CPU máy tính sẽ bị ngốn 100% ngay lập tức. Cần dùng thuật toán Exponential Backoff kết hợp Jitter.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Vòng lặp bận không nghỉ sẽ đốt cháy 100% CPU.
- Tăng dần thời gian nghỉ theo cấp số nhân (1s, 2s, 4s . . . 30s) giúp tiết kiệm tài nguyên.

```
// [SAI] Vong lap ban (Busy-Waiting) dot chay 100% CPU:
while (!connected) { Connect(); } // Khong co delay!

// [DUNG] Exponential Backoff tang dan thoi gian nghi:
await Task.Delay(TimeSpan.FromSeconds(delaySec), token);
delaySec = Math.Min(delaySec * 2, 30);
```

Listing 12: So sánh Bẫy 4: Vòng lặp bận (Busy Wait) vs Exponential Backoff

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Sai lệch định dạng thời gian UTC Zero Offset
Khi gán trường `sentAtUtc`, nếu bạn dùng `DateTime.Now.ToString()` thì thời gian sẽ mang múi giờ địa phương (+7) và thiếu chỉ định UTC zero offset ('Z' hoặc '+00:00'). Luôn dùng `DateTime.UtcNow.ToString("O")`.

5. Bộ câu hỏi vấn đáp dành riêng cho Thành viên A

Dưới đây là 20 câu hỏi trọng tâm kèm câu trả lời chi tiết giúp Thành viên A tự tin đạt điểm xuất sắc khi bảo vệ đồ án trước hội đồng:

Câu 1: Tại sao em lại chọn kích thước khung tin tối đa là 262,144 byte (256 KiB)?
Trả lời: Giới hạn 256 KiB (262,144 byte) là hoàn toàn dư dả cho các gói tin nhịp tim và báo cáo tài nguyên phần cứng (thường chỉ chiếm từ 200 byte đến vài chục kilobyte). Việc đặt giới hạn cứng này giúp máy chủ phòng vệ hiệu quả trước các cuộc tấn công từ chối dịch vụ (DoS), ngăn chặn việc cấp phát mảng byte khổng lồ làm tràn bộ nhớ RAM.

Câu 2: Tại sao độ dài khung tin phải dùng định dạng Big-Endian?
Trả lời: Big-Endian là chuẩn Network Byte Order theo quy chuẩn RFC 1700 của mạng Internet, trong đó byte có trọng số lớn nhất (MSB) được gửi trước. Điều này bảo đảm tính tương thích tuyệt đối giữa các máy tính sử dụng kiến trúc CPU khác nhau (x86 dùng Little-Endian, ARM/MIPS có thể dùng Big-Endian).

Câu 3: WireEnvelope của em có mấy trường? Kể tên?
Trả lời: Có chính xác 6 trường cố định: `version` (int = 1), `messageType` (string), `requestId` (string GUID), `deviceId` (string), `sentAtUtc` (string ISO 8601 với chỉ định UTC zero offset 'Z' hoặc '+00:00'), và `payload` (JsonElement).

Câu 4: Tại sao version lại là số nguyên 1 mà không phải chuỗi "1.0"?
Trả lời: Sử dụng kiểu số nguyên `int` giúp CPU so sánh giá trị nhanh hơn chỉ với 1 lệnh

máy, tiết kiệm băng thông mạng và tránh được các lỗi phân tích chuỗi ký tự giữa các ngôn ngữ lập trình khác nhau.

Câu 5: Tại sao Agent lại là Client mà không phải Server?
Trả lời: Agent chạy trên máy tính sinh viên, thường nằm sau tường lửa Windows Firewall và mạng NAT. Tường lửa mặc định chặn kết nối đi vào (Inbound) nhưng cho phép kết nối đi ra (Outbound). Do đó Agent phải đóng vai trò Client chủ động mở kết nối ra ngoài (Active Open) tới địa chỉ IP và cổng tĩnh của Central Server.

Câu 6: Bản tin nhịp tim gửi bao lâu một lần?
Trả lời: Bản tin `HEARTBEAT` được gửi định kỳ với chu kỳ cấu hình mặc định 5 giây.

Câu 7: Tên chuẩn của thông điệp nhịp tim là gì?
Trả lời: Tên chuẩn là `HEARTBEAT` (chiều Agent $\rightarrow$ Central) và `HEARTBEAT_ACK` (chiều Central $\rightarrow$ Agent).

Câu 8: Hiện tượng phân mảnh trong TCP là gì và em giải quyết như thế nào?
Trả lời: TCP là giao thức hướng dòng byte không có ranh giới tin nhắn. Một gói tin gửi đi có thể bị chia nhỏ thành nhiều mảnh khi truyền qua các thiết bị mạng. Em giải quyết triệt để bằng phương thức `ReadExactlyBytesAsync`: sử dụng vòng lặp `while` đọc tích lũy từng mẩu dữ liệu cho đến khi gom đủ chính xác số byte ghi trong tiêu đề 4 byte mới xử lý tiếp.

Câu 9: Thuật toán Exponential Backoff hoạt động ra sao khi mất mạng (tính năng dự kiến A-005)?
Trả lời (Kế hoạch A-005 — chưa triển khai ở hiện tại): Trong tác vụ tiếp theo (A-005), khi kết nối bị đứt, Agent sẽ chờ 1 giây trước khi thử lại. Nếu tiếp tục thất bại, thời gian chờ tăng gấp đôi: 2s, 4s, 8s, 16s và cố định ở mức trần 30s kết hợp độ trễ ngẫu nhiên (Jitter). Hiện tại ở A-003/A-004, Agent chưa có cơ chế tự động kết nối lại mà chỉ bắt lỗi và chuyển trạng thái ngắt kết nối an toàn.

Câu 10: Bộ nhớ đệm Replay Buffer giải quyết vấn đề gì (thiết kế A-006)?
Trả lời (Thiết kế A-006 — dự kiến sau A-005): Giúp duy trì tính liên tục của dữ liệu lịch sử. Khi sinh viên vô tình làm lỏng cáp mạng, các số liệu CPU/RAM được ghi nhớ trong hàng đợi của Agent và tự động phát lại tuần tự khi kết nối mạng được phục hồi.

Câu 11: Chính sách đẩy dữ liệu của Replay Buffer là gì (thiết kế A-006)?
Trả lời (Thiết kế A-006 — dự kiến sau A-005): Sử dụng cơ chế hàng đợi FIFO (First-In, First-Out), gửi từ bản ghi cũ nhất đến bản ghi mới nhất nhằm bảo đảm trình tự dòng thời gian trong cơ sở dữ liệu.

Câu 12: Điều gì xảy ra nếu bộ đệm ReplayBuffer bị đầy 50 mẻ dữ liệu (thiết kế A-006)?
Trả lời (Thiết kế A-006 — dự kiến sau A-005): Hệ thống áp dụng chính sách loại bỏ mẻ cũ nhất (`FIFO Drop-Oldest`) để nhường chỗ cho mẻ mới nhất, bảo đảm ứng dụng không bao giờ vượt quá ngưỡng tiêu thụ RAM cho phép. Lưu ý rằng chỉ mẻ số liệu (`METRIC_BATCH`) được lưu giữ kèm cờ `isReplay = true`, tuyệt đối không lưu `HEARTBEAT` hay `SERVICE_STATUS`.

Câu 13: Em đo phần trăm CPU, RAM và Uptime trên Windows bằng cách nào trong C#?
Trả lời: Trong `WindowsMetricsCollector.cs`, em sử dụng kỹ thuật Win32 P/Invoke gọi trực tiếp các API cấp thấp của Windows:

- **Tải CPU:** Gọi hàm `GetSystemTimes` lấy 2 mẫu thời gian rỗi (`idle`) và tổng thời gian (`kernel + user`), tính: $\text{CPU\%} = (1 - \frac{\Delta\text{idle}}{\Delta\text{total}}) \times 100$. Cách này đo chính xác tải của

toàn bộ hệ thống máy tính thay vì chỉ một tiến trình, đạt chuẩn WBS < 100ms (trong phiên validation 10 mẫu, max $= 0.616$ms).

- **Bộ nhớ RAM:** Gọi hàm `GlobalMemoryStatusEx` đọc trực tiếp cấu trúc `MEMORYSTATUSEX`, lấy tỷ lệ `MemoryLoad` (%) và tính RAM đang dùng: $(\text{ullTotalPhys} - \text{ullAvailPhys})/(1024 \times 1024)$ MB.
- **Uptime:** Gọi hàm `GetTickCount64()` chia cho 1000 để lấy số giây hệ điều hành hoạt động liên tục kể từ lúc bật máy.

Câu 14: Quy chuẩn định dạng thời gian sentAtUtc trong giao thức yêu cầu điều gì?
Trả lời: Giao ước mạng bắt buộc trường `sentAtUtc` phải có chỉ định độ lệch múi giờ UTC bằng 0 (UTC Zero Offset) tường minh. Các định dạng hợp lệ được chấp nhận là kết thúc bằng chữ hoa `'Z'` (ví dụ: `"2026-09-27T12:00:00Z"`) hoặc `'+00:00'`. Các chuỗi thời gian không có múi giờ, có múi giờ địa phương (như `'+07:00'`), dùng chữ thường `'z'` hoặc `'-00:00'` đều bị `EnvelopeValidator` từ chối tuyệt đối để tránh sai lệch thời gian giữa máy trạm và máy chủ.

Câu 15: Trường deviceId trong Agent hiện tại được gán như thế nào và việc đọc tự động tên máy thì sao?
Trả lời: Trong hiện trạng mã nguồn A-004, các hàm khởi tạo của `HeartbeatService` và `MetricsService` nhận tham số `deviceId` từ bên ngoài, với giá trị mặc định là `"PC-AGENT-01"` nếu không chỉ định. Việc tự động ánh xạ `Environment.MachineName` vào `deviceId` là hành vi dự kiến cho các chặng tích hợp tiếp theo (CHƯA TRIỂN KHAI ở A-004).

Câu 16: Tại sao FrameCodec lại ném lỗi khi độ dài payload bằng 0?
Trả lời: Vì một khung tin có chiều dài bằng 0 không mang nội dung nghiệp vụ, có thể xuất phát từ việc lập trình sai hoặc cố tình gửi gói tin rác làm nghẽn máy chủ.

Câu 17: Bản tin AGENT_GOODBYE và cơ chế tắt an toàn (Graceful Shutdown) hiện trạng ra sao?
Trả lời: Bản tin `AGENT_GOODBYE` đã được định nghĩa đầy đủ trong giao ước hợp đồng mạng của thư viện Protocol. Tuy nhiên, logic tự động bắt sự kiện `Console.CancelKeyPress` để gửi `AGENT_GOODBYE` trước khi thoát thuộc phạm vi tích hợp ứng dụng ở các chặng sau (CHƯA TRIỂN KHAI trong phạm vi A-003/A-004).

Câu 18: Trong thư viện Protocol, 107 bài test kiểm tra những gì?
Trả lời: 107 bài test kiểm thử bao phủ toàn diện: đóng khung nhị phân Big-Endian (35 bài), kiểm tra tính hợp lệ của WireEnvelope (28 bài), đối soát 16 fixture JSON chuẩn cùng các DTO (24 bài), và bộ xác thực EnvelopeValidator (20 bài).

Câu 19: Thành viên B và C sẽ dùng code của em như thế nào?
Trả lời: Các bạn thêm tham chiếu dự án `ProjectReference` trỏ tới `LabGuard.Protocol` vào dự án Central và Admin để dùng chung `FrameCodec`, `WireEnvelope` và các DTO.

Câu 20: Nếu xảy ra sự cố đứt cáp mạng, Agent xử lý như thế nào ở hiện tại?
Trả lời: Khi đứt cáp vật lý, hệ điều hành không gửi cờ TCP ngắt kết nối. Đến chu kỳ gửi tiếp theo, phương thức `WriteAsync` của luồng Single Writer gặp ngoại lệ socket/IO. `AgentSocketClient` lập tức xử lý lỗi: gán `_writerFaulted = true`, gọi `channel.Writer.TryComplete(ex)`, hủy `_cts` nội bộ khiến `IsConnected = false`, giúp các tác vụ đang hoặc chuẩn bị đẩy vào hàng đợi (`EnqueueFrameAsync`) lập tức thoát/ném lỗi thay vì bị treo vĩnh viễn. Hiện tại ở A-004, Agent chưa có cơ chế chuyển sang chế độ đệm (`ReplayBuffer` thuộc A-006) và chưa có tự động kết nối lại (Reconnect thuộc A-005).

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN A —